

تکلیف اول

شماره دانشجویی: 810100089

محمد امانلو

استاد سیفی پور

درس بینایی کامپیوتر

بهار 1404

سوال اول

در این بخش از پروژه‌ی پردازش تصویر، من ابتدا یک بار با استفاده از کتابخانه‌های آماده - به‌ویژه OpenCV برای تبدیل رنگ، اعمال فیلتر و افزودن نویز و ImageIO برای بارگذاری و ذخیره‌ی تصاویر JPG پیاده‌سازی‌ای سریع و قابل‌اعتماد انجام دادم؛ سپس همان مراحل را از صفر و به‌صورت دستی با کمک NumPy و SciPy (تنها برای توابع پایه‌ی عددی) دوباره پیاده‌سازی کردم تا ضمن کسب امتیاز بیشتر، درکی عمیق‌تر از الگوریتم‌های مورد‌استفاده پیدا کنم.

در نسخه‌ی اول، کار را با خواندن تصویر ورودی آغاز کردم و آن را در قالب آرایه‌ای از اعداد ذخیره نمودم. برای تولید نسخه‌ی خاکستری، از فرمول استاندارد لومیننس استفاده کردم که در آن شدت روشنایی پیکسل معادل ترکیبی خطی از مقادیر کانال‌های قرمز، سبز و آبی است. تصویر باینری هم با آستانه‌گذاری ساده روی مقدار ۱۲۷ به‌دست آمد؛ یعنی تمام پیکسل‌های بالاتر از این آستانه را سفید و بقیه را سیاه کردم. برای تغییر کانتراست، ابتدا نقطه‌ی میانه‌ی شدت خاکستری تصویر را به‌عنوان نقطه‌ی مرجع در نظر گرفتم و سپس یک تبدیلات خطی تعریف کردم که مقادیر پیکسل را بر اساس ضریب ۱.۵ (برای افزایش کانتراست) و ۰.۵ (برای کاهش کانتراست) نسبت به آن نقطه حرکت می‌دهد. این کار برای هر دو نسخه‌ی رنگی و خاکستری تصویر انجام شد و خروجی‌ها را در پوشه‌ای مشخص ذخیره کردم.

در گام بعدی، با افزودن عدد ثابت ۲۰ به هر مقدار پیکسل تصویر خاکستری و محدودسازی نتایج در بازه‌ی ۰ تا ۲۵۵، روشنایی (brightness) را افزایش دادم و تصویر حاصل را ذخیره کردم. سپس برای شبیه‌سازی نویز فلفل و نمک، به‌طور تصادفی ۲٪ از پیکسل‌ها را سفید (نمک) و ۲٪ را سیاه (فلفل) کردم تا بتوانم تأثیر نویز پراکنده بر تصویر اصلی را مشاهده کنم. همه‌ی این مراحل با توابع OpenCV و NumPy با چند خط کد خلاصه پیاده شدند و در نهایت تصاویر خروجی از جمله نسخه‌های باینری، خاکستری، رنگی، تصاویر کانتراست‌داده‌شده، روشن‌تر و نویزی را با `imageio.imwrite` در پوشه‌ی خروجی ذخیره نمودم.

با این وجود، برای عمیق‌تر کردن فهم خود از مبانی پردازش تصویر و اثبات توانایی پیاده‌سازی الگوریتم‌ها از پایه، دوباره تمام مراحل بالا را به‌صورت فرام‌اسکرچ نوشتم. در این نسخه، تصویر را همچنان با ImageIO می‌خواندم اما تبدیل به خاکستری را دستی انجام دادم: با ضرب کانال‌های RGB در ضرایب ۰/۲۹۸۹، ۰/۵۸۷۰ و ۰/۱۱۴۰ و تشکیل آرایه‌ی جدید. تولید تصویر باینری را با حلقه‌های ساده روی تک‌تک پیکسل‌ها و مقایسه با آستانه‌ی ۱۲۷ انجام دادم. تنظیم کانتراست را دقیقاً با همان فرمول خطی اما بدون استفاده از هیچ تابع آماده‌ای پیاده کردم؛ ابتدا میانه‌ی ماتریس خاکستری را محاسبه و سپس مقدار جدید هر پیکسل را بر اساس آن محاسبه و به محدوده‌ی ۰-۲۵۵ محدود کردم. برای افزایش روشنایی هم به سادگی عدد ۲۰ را به آرایه‌ی خاکستری اضافه و مجدداً clamp کردم. مرحله‌ی افزودن نویز فلفل و نمک را نیز با تولید دو آرایه‌ی تصادفی از مختصات پیکسل‌ها و تخصیص ارزش‌های ۰ و ۲۵۵ انجام دادم. نتیجه، مجموعه‌ای از تصاویر تقریباً یکسان با نسخه‌ی کتابخانه‌ای بود، اما با این تفاوت که من دقیقاً می‌دانستم هر خط کد چه کاری انجام می‌دهد و چرا تأثیرگذار است.

سوال (2)

در این بخش از پروژه، ابتدا تصمیم گرفتم دو رویکرد مختلف را برای پردازش تصویر پیاده‌سازی کنم: یک بار با استفاده از کتابخانه‌های آماده و معروف مانند OpenCV و بار دیگر بدون استفاده از هیچ کتابخانه‌ای جز NumPy و SciPy. هدف از این کار این بود که علاوه بر استفاده از ابزارهای آماده برای تسریع در انجام پروژه، عمق بیشتری از نحوه کار الگوریتم‌های پردازش تصویر و چگونگی پردازش‌های پایه‌ای تصویر مانند تبدیل رنگ‌ها، هیستوگرام‌ها و تبدیل فوری به دست بیاورم. در این گزارش به تفصیل به توضیح هر دو روش پرداخته‌ام و نتایج و تحلیل‌هایی را که از این روش‌ها به دست آمده‌اند، بررسی کرده‌ام.

در گام اول، برای پردازش تصویر از کتابخانه‌های آماده استفاده کردم. این روش به من این امکان را داد که در مدت زمان کمتری تصویر را پردازش کنم و از ابزارهای استاندارد بهره ببرم. ابتدا تصویری که در تمرین قبلی ذخیره کرده بودم را با استفاده از کتابخانه ImageIO لود کردم. این کتابخانه قابلیت خواندن و نوشتن تصاویر را با فرمت‌های مختلف مانند JPG و PNG دارد و من از آن برای بارگذاری تصویر در قالب آرایه‌ای از اعداد استفاده کردم. پس از بارگذاری تصویر، آن را به سه مدل رنگی مختلف تقسیم کردم؛ مدل‌های RGB (رنگی)، HSV (مناسب برای تجزیه رنگ‌ها) و CMY (مدل مکمل رنگ‌ها).

در مرحله بعد، تصویر را به‌صورت خاکستری تبدیل کردم. این کار با استفاده از فرمول استاندارد برای تبدیل رنگ‌ها به خاکستری (که وزن‌هایی به مقادیر قرمز، سبز و آبی اختصاص می‌دهد) انجام شد.

سپس تصویری باینری ایجاد کردم که در آن پیکسل‌های بالاتر از مقدار آستانه ۱۲۷ سفید و پیکسل‌های پایین‌تر سیاه شدند. این کار به‌منظور برجسته کردن نواحی با کنتراست بالا در تصویر انجام شد.

پس از آن، هیستوگرام تصویر خاکستری و رنگی را با استفاده از توابع آماده‌ای که OpenCV و Matplotlib ارائه می‌دهند رسم کردم. این هیستوگرام‌ها نشان‌دهنده‌ی توزیع شدت‌های روشنایی در تصویر بودند و به من این امکان را دادند که کنتراست تصاویر را تحلیل کنم. برای تحلیل کنتراست، هیستوگرام‌ها را مقایسه کردم و متوجه شدم که تصویر رنگی از کنتراست بالاتری برخوردار است. در مرحله بعد، هیستوگرام تصویر خاکستری را با استفاده از روش هموارسازی (Smoothing) هموار کردم تا نویزهای موجود در آن کاهش یابد و تغییرات اصلی بهتر دیده شوند. این هموارسازی به‌وسیله میانگین متحرک با اندازه هسته ۵ انجام شد.

در نهایت، تبدیل فوریه تصویر خاکستری را با استفاده از توابع FFT (تبدیل فوریه سریع) در NumPy انجام دادم. این تبدیل به من این امکان را داد که محتویات فرکانسی تصویر را بررسی کنم و اجزای تصویر مانند لبه‌ها و بافت‌ها را تحلیل کنم. نتیجه‌ی این تبدیل، نمایی از فرکانس‌های پایین و بالا بود که به‌خوبی ساختار و بافت تصویر را نشان می‌داد.

در گام بعدی، به‌منظور یادگیری عمیق‌تر فرآیندهای پردازش تصویر و به‌دست آوردن درک دقیق‌تری از نحوه عملکرد الگوریتم‌ها، تصمیم گرفتم تمام مراحل بالا را بدون استفاده از هیچ‌یک از کتابخانه‌های آماده (مانند OpenCV) پیاده‌سازی کنم و فقط از NumPy و SciPy برای انجام محاسبات عددی استفاده کنم. این کار باعث شد که بیشتر با جزئیات الگوریتم‌ها و نحوه‌ی پردازش داده‌های تصویر آشنا شوم و عملاً خودم به‌صورت گام به گام تمامی مراحل را پیاده‌سازی کنم.

برای این کار، ابتدا همانند روش اول، تصویر را با استفاده از ImageIO بارگذاری کردم و سپس آن را به‌صورت دستی به خاکستری تبدیل کردم. این تبدیل با ضرب دادن مقادیر RGB به وزن‌های خاص برای هر کانال و سپس جمع آن‌ها انجام شد. سپس برای تولید تصویر باینری، به‌صورت دستی روی تک‌تک پیکسل‌ها مقایسه با آستانه ۱۲۷ انجام دادم و پیکسل‌های بالاتر از این مقدار را سفید و پیکسل‌های پایین‌تر را سیاه کردم.

در مرحله بعد، به‌منظور تحلیل کنتراست، خودم تابعی را نوشتم که برای تنظیم کنتراست، مقدار میانه شدت‌های تصویر خاکستری را به‌عنوان نقطه مرجع در نظر می‌گیرد و با استفاده از یک تبدیل خطی، مقادیر پیکسل‌ها را نسبت به این نقطه افزایش یا کاهش می‌دهد. این مرحله به‌صورت دستی پیاده‌سازی شد و دقیقاً مطابق با روش اول به نتیجه رسیدم.

در مراحل بعدی، برای هموارسازی هیستوگرام، از یک میانگین متحرک استفاده کردم. این روش به صورت دستی انجام شد و به من این امکان را داد که نویز موجود در هیستوگرام را کاهش دهم و تنها تغییرات اصلی را مشاهده کنم.

در نهایت، تبدیل فوریه تصویر خاکستری را به طور دستی و با استفاده از توابع پایه‌ای NumPy پیاده‌سازی کردم. با استفاده از تبدیل فوریه سریع (FFT) توانستم فرکانس‌های تصویر را استخراج کرده و آن‌ها را نمایش دهم. نتیجه‌ی این تبدیل نیز مشابه همان چیزی بود که در استفاده از کتابخانه‌ها به دست آمده بود.

پس از انجام هر دو پیاده‌سازی، نتایج بسیار مشابه بودند، با این تفاوت که در پیاده‌سازی دستی، توانستم کنترل بیشتری روی جزئیات الگوریتم‌ها داشته باشم و دقیقاً بفهمم که هر مرحله از پردازش تصویر چگونه انجام می‌شود. در هر دو روش، تصاویر خاکستری، باینری و رنگی به خوبی تولید شدند و هیستوگرام‌های آن‌ها نیز قادر به نشان دادن میزان کنتراست و توزیع شدت‌های روشنایی در تصویر بودند.

اما یکی از نکات مهم این بود که در پیاده‌سازی با کتابخانه‌ها، سرعت انجام کار به طور قابل توجهی بالاتر بود. این موضوع به ویژه در پردازش‌های پیچیده مانند تبدیل فوریه و هموارسازی هیستوگرام خود را نشان داد، جایی که با استفاده از کتابخانه‌ها تمامی این مراحل در چند خط کد انجام می‌شد، در حالی که در پیاده‌سازی دستی، نیاز به نوشتن کدهای بیشتر برای هر مرحله داشتیم.

در نهایت، می‌توان گفت که استفاده از کتابخانه‌ها برای پروژه‌های بزرگ و صنعتی مناسب‌تر است، زیرا به سرعت کار می‌آید و امکان تمرکز بیشتر روی تحلیل و نتایج نهایی را فراهم می‌آورد. اما در پروژه‌های آموزشی یا تحقیقاتی، پیاده‌سازی دستی می‌تواند به درک بهتر الگوریتم‌ها و مفاهیم پایه‌ای کمک کند و باعث افزایش دانش فردی شود.

سوال (3)

در این بخش از پروژه، هدف من تحلیل و بررسی فرمت تصویر ورودی به طور کامل از نظر هدر، بیت استریم، رزولوشن، نوع فشرده‌سازی و الگوریتم فشرده‌سازی آن بود. این تحلیل‌ها به من کمک کرد که جزئیات دقیقی از نحوه ذخیره‌سازی تصویر و نحوه فشرده‌سازی آن به دست آورم. برای انجام این تحلیل‌ها، ابتدا با استفاده از کتابخانه‌ی ImageIO تصویر را لود کرده و سپس ویژگی‌های مختلف آن را استخراج و بررسی کردم.

ابتدا تصویر را با استفاده از کتابخانه‌ی ImageIO بارگذاری کردم. این کتابخانه به من این امکان را می‌دهد که تصویر را به صورت یک آرایه NumPy درآورم که به راحتی بتوانم پیکسل‌ها را دستکاری کرده و ویژگی‌های مختلف تصویر را استخراج کنم. تصویر ورودی به فرمت JPG بوده و آن را در محیط پایتون بارگذاری کردم. این تصویر سپس به صورت آرایه‌ای از داده‌های عددی تبدیل می‌شود که می‌توانم روی آن عملیات مختلفی انجام دهم.

مرحله‌ی بعدی تحلیل هدر تصویر بود. در این بخش، باید می‌فهمیدم که تصویر تحت کدام فرمت ذخیره شده است. به‌طور کلی، هر فرمت تصویری مانند JPEG، PNG یا BMP یک هدر خاص دارد که اطلاعات مربوط به فرمت فایل و ویژگی‌های آن را در بر می‌گیرد. برای استخراج هدر تصویر، فایل را به‌صورت باینری باز کردم و اولین 8 بایت آن را خواندم. برای تصویر JPG، این 8 بایت به‌دست آمده عبارت بود از `'b\xff\xd8\xff\xe0\x00\x10JF'`. این بیت‌ها مشخص می‌کنند که تصویر از فرمت JPEG استفاده می‌کند. در فرمت JPEG، بایت‌های `0xFF 0xD8` نشان‌دهنده شروع فایل و `0xFF 0xE0` نشان‌دهنده شروع APP0 است که حاوی متادیتا برای فایل‌های JPEG است. این اطلاعات هدر به وضوح نشان‌دهنده این هستند که تصویر در فرمت JPEG ذخیره شده است.

پس از بررسی هدر، به استخراج رزولوشن تصویر پرداختم. رزولوشن تصویر نشان‌دهنده ابعاد تصویر در دو بعد عرض و ارتفاع است. برای استخراج این اطلاعات، از ویژگی `img.shape` استفاده کردم که ابعاد تصویر را به صورت یک آرایه به من می‌دهد. برای تصویر بارگذاری شده، ابعاد تصویر به این شکل بود:

● عرض: 960 پیکسل و ارتفاع: 1280 پیکسل

این ابعاد نشان می‌دهند که تصویر به صورت عمودی (portrait) است. دانستن رزولوشن تصویر برای انجام کارهای مختلفی مانند فشرده‌سازی، تغییر مقیاس و تحلیل تصویر اهمیت زیادی دارد.

در مرحله بعد، مهم‌ترین بخش تحلیل یعنی نوع فشرده‌سازی تصویر را بررسی کردیم. نوع فشرده‌سازی تصویر، مشخص می‌کند که آیا تصویر فشرده شده است یا خیر و اگر فشرده شده، از چه الگوریتمی برای این کار استفاده شده است. در اینجا، تصویر ورودی از نوع JPEG است که از نوع فشرده‌سازی از دست دادن اطلاعات (lossy) است. در فشرده‌سازی JPEG، برخی از داده‌های تصویر که از نظر انسان‌ها قابل تشخیص نیستند، حذف می‌شوند تا حجم فایل کاهش یابد. این نوع فشرده‌سازی با استفاده از تکنیک‌هایی مانند DCT (تبدیل کسینوس گسسته) انجام می‌شود که فرکانس‌های بالای تصویر (که معمولاً به جزئیات و نویز مربوط می‌شوند) را حذف کرده و فقط فرکانس‌های پایین‌تر (که به ساختار کلی تصویر مربوط هستند) را حفظ می‌کند.

این فشرده‌سازی به‌گونه‌ای است که مقداری از کیفیت تصویر کاهش پیدا می‌کند، اما حجم فایل به‌شدت کاهش می‌یابد. فشرده‌سازی JPEG به‌خوبی تعادل بین کیفیت و حجم فایل را برقرار می‌کند و به همین دلیل یکی از فرمت‌های پرکاربرد برای ذخیره‌سازی تصاویر است. این فرمت در عین حال که حجم فایل را کاهش می‌دهد، کیفیت تصویر را به‌گونه‌ای حفظ می‌کند که برای بسیاری از کاربردها مانند استفاده در وب و ذخیره‌سازی تصاویر در عکاسی و صنایع مختلف مناسب باشد.

در بخش بعدی به تحلیل بیت استریم تصویر پرداختم. بیت استریم تصویر به مجموعه‌ای از داده‌های باینری اطلاق می‌شود که فایل تصویر را تشکیل می‌دهد. این داده‌ها شامل هدر، داده‌های پیکسل و متادیتا می‌باشند. در اینجا، بیت استریم تصویر JPEG به‌گونه‌ای کدگذاری شده است که داده‌های پیکسل به‌صورت فشرده ذخیره شده‌اند و متادیتا در هدر قرار دارد. برای تحلیل دقیق‌تر بیت استریم، باید فرمت JPEG را به‌طور کامل بررسی کرد، اما در این گزارش به‌طور کلی می‌توان گفت که بیت استریم تصویر شامل هدر JPEG و داده‌های فشرده‌شده پیکسل‌ها است که به‌طور قابل توجهی حجم فایل را کاهش می‌دهد.

- فرمت تصویر: تصویر ورودی در فرمت JPEG ذخیره شده است که هدر آن با بایت‌های `'b'\xff\xd8\xff\xe0\x00\x10JF` تأیید می‌شود.
- رزولوشن: ابعاد تصویر 960x1280 پیکسل است که به‌طور عمودی (portrait) بودن آن اشاره دارد.
- نوع فشرده‌سازی: تصویر از فشرده‌سازی JPEG استفاده می‌کند که نوعی فشرده‌سازی از دست دادن اطلاعات است. این نوع فشرده‌سازی باعث کاهش حجم تصویر و کاهش جزئیات آن می‌شود.
- بیت استریم: بیت استریم این فایل شامل هدر JPEG و داده‌های فشرده‌شده پیکسل‌ها است که با استفاده از الگوریتم فشرده‌سازی JPEG کدگذاری شده‌اند.

سوال (4)

در این بخش از پروژه، من به مقایسه تصاویر مختلف چهره خود پرداختم که شامل چهار تصویر از چهره خودم بود. هر یک از این تصاویر از زاویه‌ها و فواصل مختلف گرفته شده بودند و هدف این بود که شباهت میان این تصاویر را با استفاده از تکنیک‌های تطبیق الگو بررسی کنم. در ابتدا، از روش‌های مختلفی برای مقایسه این تصاویر استفاده کردم و سپس نتایج حاصل را تحلیل کردم. این روش‌ها شامل تطبیق الگو با استفاده از هم‌میزان‌سازی مقاطع (Normalized Cross-Correlation)، مقایسه

هیستوگرام‌ها، محاسبه خطای مربع میانگین (MSE) و همچنین استفاده از شاخص شباهت ساختاری (SSIM) بودند. در ادامه، به‌طور مفصل به شرح مراحل انجام این تحلیل‌ها، نتایج آن‌ها و تجزیه و تحلیل هر یک پرداخته‌ام.

اولین گام در این پروژه لود کردن تصاویر بود. تصاویر مورد نظر من شامل چهار تصویر مختلف از چهره‌ام بود که به شرح زیر هستند:

- تصویر اول: تصویری از چهره به‌طور مستقیم (نمای روبرو).
- تصویر دوم: تصویری دیگری از چهره‌ام با همان زاویه اما از فاصله نزدیک‌تر.
- تصویر سوم: تصویری از چهره که در زاویه 45 درجه به دوربین نگاه می‌کنم.
- تصویر چهارم: تصویری که بخشی از چهره‌ام توسط دست یا ماسک پوشانده شده است.

تمام این تصاویر با فرمت JPG و با استفاده از کتابخانه ImageIO لود شدند. برای اطمینان از یکپارچگی مقایسه‌ها، ابتدا همه تصاویر به صورت خاکستری (grayscale) تبدیل شدند. این کار به‌ویژه مهم بود زیرا تجزیه و تحلیل تطبیق الگو و شباهت ساختاری اغلب روی تصاویر تک‌کاناله (مانند تصاویر خاکستری) با دقت بیشتری انجام می‌شود.

تطبیق الگو با استفاده از هم‌میزان‌سازی مقاطع (NCC)

در مرحله بعد، از تطبیق الگو با استفاده از معیار هم‌میزان‌سازی مقاطع (Normalized Cross-Correlation یا NCC) برای مقایسه شباهت تصاویر استفاده کردم. این روش با محاسبه هم‌میزان‌سازی نرمال شده بین تصویر مرجع (تصویر اول) و تصاویر دیگر، میزان شباهت هر تصویر به تصویر اول را مشخص می‌کند. نتایج به‌دست آمده نشان داد که:

- تصویر دوم (که از همان زاویه است اما از فاصله نزدیک‌تر گرفته شده) شباهت بالایی با تصویر اول داشت. این نتیجه نشان می‌دهد که تغییر فاصله تأثیر زیادی بر شباهت الگو ندارد، زیرا ویژگی‌های اصلی چهره حفظ شده‌اند.
- تصویر سوم (زاویه 45 درجه) شباهت کمتری نسبت به تصویر اول داشت. این امر به‌دلیل تغییرات در ویژگی‌های صورت به‌دلیل زاویه‌دار بودن است که باعث کاهش تطبیق الگو می‌شود.
- تصویر چهارم (صورت پوشیده با ماسک) کمترین شباهت را داشت. این نتیجه منطقی است چرا که بخشی از صورت پوشانده شده و این مسئله باعث می‌شود که تطبیق الگو کاهش یابد.

تحلیل هیستوگرام‌ها

در مرحله بعد، به بررسی هیستوگرام‌های تصاویر پرداختم. هیستوگرام‌ها به‌طور مؤثر توزیع شدت پیکسل‌ها را در تصاویر نشان می‌دهند و می‌توانند تفاوت‌های روشنایی و کنتراست میان تصاویر را مشخص کنند. از آنجا که تصاویر رنگی در این مرحله به تصاویر خاکستری تبدیل شده بودند، من هیستوگرام تصویر خاکستری (تصویر اول) را با هیستوگرام تصاویر دیگر مقایسه کردم. نتایج به‌دست آمده نشان داد:

- تصویر دوم (که از همان زاویه و با فاصله نزدیک‌تر است) بیشترین شباهت را با تصویر اول داشت. این به‌خاطر شباهت زیاد در توزیع شدت پیکسل‌ها است.
- تصویر سوم (زاویه 45 درجه) دارای هیستوگرام متفاوت‌تری بود و تغییرات در زاویه تصویر باعث تغییراتی در توزیع روشنایی و سایه‌ها شد.
- تصویر چهارم (پوشیده با ماسک) نیز تفاوت قابل‌توجهی با تصویر اول داشت. بخش‌های پوشیده‌شده باعث تغییر در توزیع شدت‌های پیکسل‌ها و در نتیجه کاهش شباهت شدند.

محاسبه خطای مربع میانگین (MSE)

در این بخش، من از محاسبه خطای مربع میانگین (MSE) برای اندازه‌گیری شباهت میان تصاویر استفاده کردم. MSE با محاسبه میانگین مربع تفاوت‌ها میان مقادیر پیکسل‌های دو تصویر، تفاوت‌ها را نشان می‌دهد. نتایج حاصل از MSE نشان داد:

- تصویر دوم کمترین MSE را داشت که به معنی شباهت بالا بین تصویر دوم و تصویر اول است.
- تصویر سوم MSE بالاتری داشت که نشان‌دهنده تفاوت بیشتر به‌دلیل تغییر زاویه است.
- تصویر چهارم نیز MSE نسبتاً بالاتری داشت که به‌دلیل پوشش بخش‌هایی از صورت با ماسک بود.

تحلیل شاخص شباهت ساختاری (SSIM)

در نهایت، من از شاخص شباهت ساختاری (SSIM) برای مقایسه ساختاری تصاویر استفاده کردم. SSIM یک معیار جدیدتر است که علاوه بر شدت روشنایی، به تفاوت‌های ساختاری و کنتراست نیز توجه می‌کند. نتایج SSIM به‌طور خلاصه به شرح زیر بود:

- تصویر دوم نزدیک‌ترین شباهت را به تصویر اول داشت، اما تفاوت‌هایی به‌دلیل تغییر فاصله وجود داشت که باعث کاهش امتیاز SSIM شد.

تصویر سوم بیشترین امتیاز SSIM را داشت که نشان‌دهنده حفظ ساختار اصلی چهره در زاویه 45 درجه است.

- تصویر چهارم به دلیل پوشش بخشی از صورت، امتیاز کمتری در SSIM داشت

سوال (5)

در این پروژه، من به تحلیل و شناسایی ضایعات مغزی در تصاویر MRI پرداختم. این تصاویر از یک مجموعه MRI که در آن‌ها تصاویر مختلف مغز بیماران در حالت‌های متفاوت قرار داشت، استفاده کردم. هدف من این بود که با استفاده از تکنیک‌های پردازش تصویر و مقایسه هیستوگرام نیمکره‌های چپ و راست مغز، وجود ضایعه مغزی را شناسایی کنم. در این راستا، من مراحل مختلفی را طی کردم که در ادامه به تفصیل توضیح می‌دهم.

در ابتدا، تصاویر MRI را لود کردم و آن‌ها را به تصاویر خاکستری تبدیل کردم. این کار برای ساده‌سازی محاسبات و تمرکز بیشتر روی اطلاعات شدت روشنایی انجام شد. بعد از تبدیل تصاویر به خاکستری، هیستوگرام هر نیمکره چپ و راست تصاویر را محاسبه کردم. هیستوگرام‌ها نشان‌دهنده توزیع شدت پیکسل‌ها در تصویر هستند و می‌توانند تفاوت‌های موجود بین نیمکره‌ها را در صورتی که ضایعه‌ای در یکی از آن‌ها وجود داشته باشد، آشکار کنند.

تکنیک Histogram Matching

برای مقایسه و تطبیق هیستوگرام‌ها از تکنیک Histogram Matching استفاده کردم. در این تکنیک، ابتدا هیستوگرام هر یک از نیمکره‌ها (چپ و راست) را محاسبه کرده و با یکدیگر مقایسه کردم. سپس با استفاده از این مقایسه‌ها، تغییرات شدت روشنایی در هر یک از نیمکره‌ها را تحلیل کردم. به این ترتیب، قادر به شناسایی تفاوت‌های موجود بین نیمکره‌ها شدم. به کمک یک مقدار آستانه مناسب (1200) برای عدم شباهت، توانستم وجود ضایعه مغزی را تشخیص دهم.

نتایج حاصل از مقایسه تصاویر

- تصویر 4: در این تصویر، با مقدار دیسیمیلازیتی 1392.74 مواجه شدم. این مقدار نشان‌دهنده عدم تقارن قابل‌توجه بین دو نیمکره است. اگرچه این مقدار نسبت به دیگر تصاویر کمتر بود، اما هنوز از آستانه تعیین‌شده بیشتر بود، که نشان‌دهنده وجود ضایعه مغزی در تصویر است.

این ضایعه ممکن است کوچکتر یا کمتر تهاجمی باشد، اما همچنان تفاوت‌های قابل‌توجهی بین نیمکره‌ها وجود دارد.

- تصویر 3: مقدار دیسیمیلاریتی این تصویر 1506.16 بود که نشان‌دهنده تفاوت بیشتری نسبت به تصویر قبلی است. این مقدار بالاتر به این معنا است که شدت تغییرات در نیمکره‌ها بیشتر است و احتمالاً ضایعه‌ای بزرگتر یا تهاجمی‌تر در یکی از نیمکره‌ها وجود دارد. تغییرات در هیستوگرام تصویر، که به وضوح در تفاوت‌های توزیع شدت‌های پیکسل‌ها نمایان است، نشان‌دهنده وجود چنین ضایعه‌ای است.

- تصویر 2: در این تصویر، دیسیمیلاریتی 1668.51 بود که نشان‌دهنده تغییرات بسیار قابل‌توجهی بین نیمکره‌ها است. این تغییرات به احتمال زیاد به وجود ضایعه‌ای بزرگتر و تهاجمی‌تر در یکی از نیمکره‌ها مربوط می‌شود. ضایعه به طور واضح تأثیر زیادی روی توزیع شدت پیکسل‌ها در نیمکره دچار آسیب گذاشته است.

- تصویر 5: در این تصویر، دیسیمیلاریتی 1665.91 بود که مشابه تصویر 2 است. این شباهت در مقادیر دیسیمیلاریتی نشان‌دهنده این است که درجه عدم تقارن و تأثیر ضایعه بین این دو تصویر مشابه است. این نتیجه نشان می‌دهد که متد مورد استفاده در شناسایی ضایعه‌ها همخوانی و قابلیت اطمینان خوبی دارد.

- تصویر 6: مقدار دیسیمیلاریتی این تصویر 2110.49 بود که بالاترین مقدار در میان این تصاویر است. این مقدار بسیار بالا نشان‌دهنده تغییرات بسیار زیادی بین نیمکره‌ها است، که به احتمال زیاد ضایعه‌ای بسیار بزرگ یا شدید وجود دارد. این ضایعه ممکن است بخش‌های زیادی از یک نیمکره را تحت تأثیر قرار داده و به طور قابل‌توجهی توزیع شدت پیکسل‌ها را تغییر داده باشد.

- تصویر 7: دیسیمیلاریتی این تصویر 2176.84 بود که بالاترین مقدار در میان تصاویر مختلف بود. این مقدار نشان‌دهنده تفاوت بسیار زیاد بین نیمکره‌ها است و به وضوح وجود یک ضایعه بزرگ و شدید را تأیید می‌کند که ساختار مغز را به‌طور چشمگیری تحت تأثیر قرار داده است.

- تصویر 8: دیسیمیلاریتی این تصویر 1285.70 بود که کمترین مقدار در میان تصاویر بود، اما هنوز هم بالاتر از آستانه تعیین‌شده بود. این مقدار نسبتاً پایین‌تر ممکن است نشان‌دهنده این باشد که ضایعه در این تصویر نسبت به دیگر تصاویر کوچکتر یا کمتر تهاجمی است، اما همچنان وجود آن تأیید می‌شود.

شناسایی و بایندینگ با استفاده از Segmentation

بعد از شناسایی عدم تقارن، برای برجسته‌سازی نواحی آسیب‌دیده از Segmentation استفاده کردم. ابتدا تصویر را با استفاده از یک Gaussian Blur دستی برای کاهش نویز و حذف تغییرات جزئی، صاف کردم. سپس با استفاده از آستانه‌گذاری دستی، تصویر را به یک تصویر دودویی تبدیل کردم که در آن پیکسل‌های بالاتر از آستانه سفید و بقیه سیاه شدند. این کار باعث شد که نواحی غیرعادی در تصویر مشخص شوند.

بعد از اعمال آستانه‌گذاری، با استفاده از تحلیل Connected Components نواحی متصل را شناسایی کرده و برای هر منطقه‌ی بزرگ، Bounding Box کشیدم تا محل ضایعه مغزی را نشان دهم. این کار باعث شد که نواحی آسیب‌دیده به وضوح در تصویر اصلی شبیه‌سازی و علامت‌گذاری شوند.